ClassSync — Documentação dos testes da API (Postman)

Objetivo e escopo

A coleção **ClassSync API** tem 56 requisições em 5 pastas e comprova que a API REST do ClassSync autentica, autoriza por perfil, valida os dados e persiste o fluxo completo do acompanhamento bimestral.

- **Autenticação e autorização:** login com JWT, rotas protegidas (401 sem token) e permissões por perfil (403 para Professor, Admin ou Responsável fora do seu papel).
- **Validação:** dados obrigatórios, formatos e regras de negócio retornam 400; duplicidades retornam 409; recursos de outra família retornam 404.
- **Fluxo do acompanhamento:** rascunho → enviado → devolvido → reenviado → publicado → ciência do responsável.
- **Persistência:** cada teste lê e grava no banco MySQL por meio do backend Node/Express.

Os testes rodam no ambiente **local** (backend em `localhost:3000` e banco local com a carga de demonstração). Os testes que criam ou apagam dados não são executados em produção.

Ambiente e como executar

Os dois arquivos ficam no repositório, em `documentos/Entrega 2/Desenvolvimento Web Fullstack/`: a coleção `ClassSync API.postman_collection_final.json` e o environment `ClassSync.postman_environment.json`.

1. **Preparar o banco local:** rodar o `database/schema.sql`, depois `node src/scripts/criarAdmin.js` e `node src/scripts/cargaDemonstracao.js`. Alguns testes usam IDs da carga (ex.: aluno 17, alunos 1 e 5).
2. **Subir o backend:** `npm run dev` na pasta `Backend`, com o `.env` local.
3. **Importar no Postman:** coleção e environment; selecionar o environment **ClassSync** (no Postman de quem executou, renomeado para ClassSync Local) (`baseUrl` = `http://localhost:3000/api`).
4. **Preencher as senhas:** `senhaAdmin`, `senhaProfessor` e `senhaResponsavel` são variáveis do tipo *secret*, preenchidas por cada integrante com as senhas do próprio `.env`. Nenhuma senha aparece neste documento nem no repositório.
5. **Executar em ordem:** a pasta **Auth** roda primeiro, porque os logins guardam os tokens usados pelas outras pastas. A pasta **Ciência** roda por último.
6. **Rodar tudo de uma vez:** botão direito na coleção → **Run collection** (Collection Runner), mantendo a ordem padrão.

Teste automático de status. O status esperado está no nome de cada requisição, por exemplo `Login - senha errada (401)`. O script abaixo fica na coleção (**Scripts → Post-response**), roda depois de toda requisição e compara o status recebido com o do nome:

```
// Status esperado vem do nome da requisição: "Criar duplicado (409)" → 409
const esperado = pm.info.requestName.match(/\((\d{3})\)\s*$/);
```

```
if (esperado) {
  pm.test(`Status ${esperado[1]}`, () => {
    pm.response.to.have.status(Number(esperado[1]));
  });
}

// Toda resposta da API deve chegar em menos de 3 segundos
pm.test("Resposta em menos de 3 s", () => {
  pm.expect(pm.response.responseTime).to.be.below(3000);
});
```

Resumo da execução

Pasta	Requisições	Aprovadas	Reprovadas
Auth	12	12	0
Catálogo	4	4	0
Aluno	11	11	0
Acompanhamento	23	23	0
Ciência	6	6	0
Total	**56**	56	0

As 56 requisições somam **115 verificações**: 2 por requisição (status esperado e tempo de resposta) e 3 testes extras da pasta Ciência. Todas foram aprovadas.

Data da execução: 09/11/2026, 15:41 · Executado por: Matheus Gajewski de Melo

Casos de teste

Cada tabela segue a ordem de execução da pasta. A coluna **Esperado** é o status HTTP verificado pelo teste automático.

Auth

#	Requisição	Método e rota	Perfil	Esperado	O que comprova
1	Login Admin	POST /auth/login	Admin	200	Login correto devolve o token JWT, guardado para os próximos testes
2	Login Professor	POST /auth/login	Professor	200	Idem, para o professor
3	Login Responsável	POST /auth/login	Responsável	200	Idem, para o responsável
4	Login - senha errada	POST /auth/login	—	401	Senha incorreta não entra e não revela se o e-

#	Requisição	Método e rota	Perfil	Esperado	O que comprova
					mail existe
5	Cadastrar Usuário	POST /auth/usuarios	Admin	201	Admin cria usuário; a senha é gravada como hash (bcrypt)
6	Cadastrar Usuário - sem token	POST /auth/usuarios	—	401	Rota protegida exige token
7	Cadastrar Usuário - professor	POST /auth/usuarios	Professor	403	Só o Admin cadastra usuários
8	Cadastrar Usuário - e-mail repetido	POST /auth/usuarios	Admin	400	E-mail é único no sistema
9	Listar Usuários	GET /auth/usuarios	Admin	200	Lista sem expor o hash da senha
10	Listar só professores	GET /auth/usuarios?perfil=PROFESSOR	Admin	200	Filtro por perfil
11	Listar perfil inválido	GET /auth/usuarios?perfil=XYZ	Admin	400	Filtro aceita só perfis válidos
12	Professor Listar Usuários	GET /auth/usuarios	Professor	403	Lista de usuários é restrita ao Admin

Catálogo

#	Requisição	Método e rota	Perfil	Esperado	O que comprova
13	Listar Tags	GET /tags	Autenticado	200	Tags disponíveis para os acompanhamentos
14	Listar Disciplinas	GET /disciplinas	Autenticado	200	Disciplinas com código e área
15	Listar Turmas	GET /turmas	Autenticado	200	Turmas com série, turno e ano letivo
16	Listar	GET	Autenticado	200	Períodos com datas de

#	Requisição	Método e rota	Perfil	Esperado	O que comprova
	Bimestres	/bimestres			abertura e encerramento

Aluno

#	Requisição	Método e rota	Perfil	Esperado	O que comprova
17	Admin Listando Alunos	GET /alunos	Admin	200	Admin vê todos os alunos
18	Professor Listar Alunos	GET /alunos	Professor	200	Professor consulta alunos
19	Responsável Listar Alunos	GET /alunos	Responsável	200	Responsável vê só os alunos vinculados a ele
20	Responsável Listar Alunos Por ID	GET /alunos/5	Responsável	200	Acesso ao próprio aluno
21	Responsável Listar Aluno de outra família	GET /alunos/1	Responsável	404	Aluno de outra família fica invisível (LGPD)
22	Buscar Aluno por ID	GET /alunos/3	Admin	200	Busca individual
23	Listar Alunos - sem token	GET /alunos	—	401	Rota protegida exige token
24	Criar Aluno - professor	POST /alunos	Professor	403	Só o Admin cadastra alunos
25	Criar Aluno - admin	POST /alunos	Admin	201	Cadastro grava no banco e devolve o id
26	Atualizar Aluno	PUT /alunos/{id}	Admin	200	Edição do aluno criado no teste 25
27	Excluir Aluno	DELETE /alunos/{id}	Admin	200	Exclusão limpa o dado de teste

Acompanhamento

#	Requisição	Método e rota	Perfil	Esperado	O que comprova
28	Listar Acompanham	GET /acompanhamentos/	Professor	200	Consulta de um

#	Requisição	Método e rota	Perfil	Esperado	O que comprova
	entos	1			registro
29	Criar Bimestre Fechado	POST /acompanhamentos	Professor	400	Não se registra fora do período aberto
30	Tag que não existe	POST /acompanhamentos	Professor	400	Tags precisam existir no catálogo
31	Criar sem vínculo	POST /acompanhamentos	Professor	403	Professor só registra na turma e disciplina que leciona
32	Criar - admin	POST /acompanhamentos	Admin	403	Registro é papel do professor
33	Criar - professor	POST /acompanhamentos	Professor	201	Cria o rascunho e guarda o id para o fluxo
34	Criar duplicado	POST /acompanhamentos	Professor	409	Um registro por aluno, disciplina e bimestre
35	Buscar por ID	GET /acompanhamentos/{id}	Professor	200	Lê o rascunho criado
36	Editar Rascunho	PUT /acompanhamentos/{id}	Professor	200	Rascunho pode ser editado
37	Enviar para revisão	PATCH /acompanhamentos/{id}/enviar	Professor	200	Rascunho passa a Enviado
38	Editar Rascunho depois de enviado	PUT /acompanhamentos/{id}	Professor	400	Registro enviado fica travado para edição
39	Professor Devolvendo	PATCH /acompanhamentos/{id}/devolver	Professor	403	Só o Admin devolve

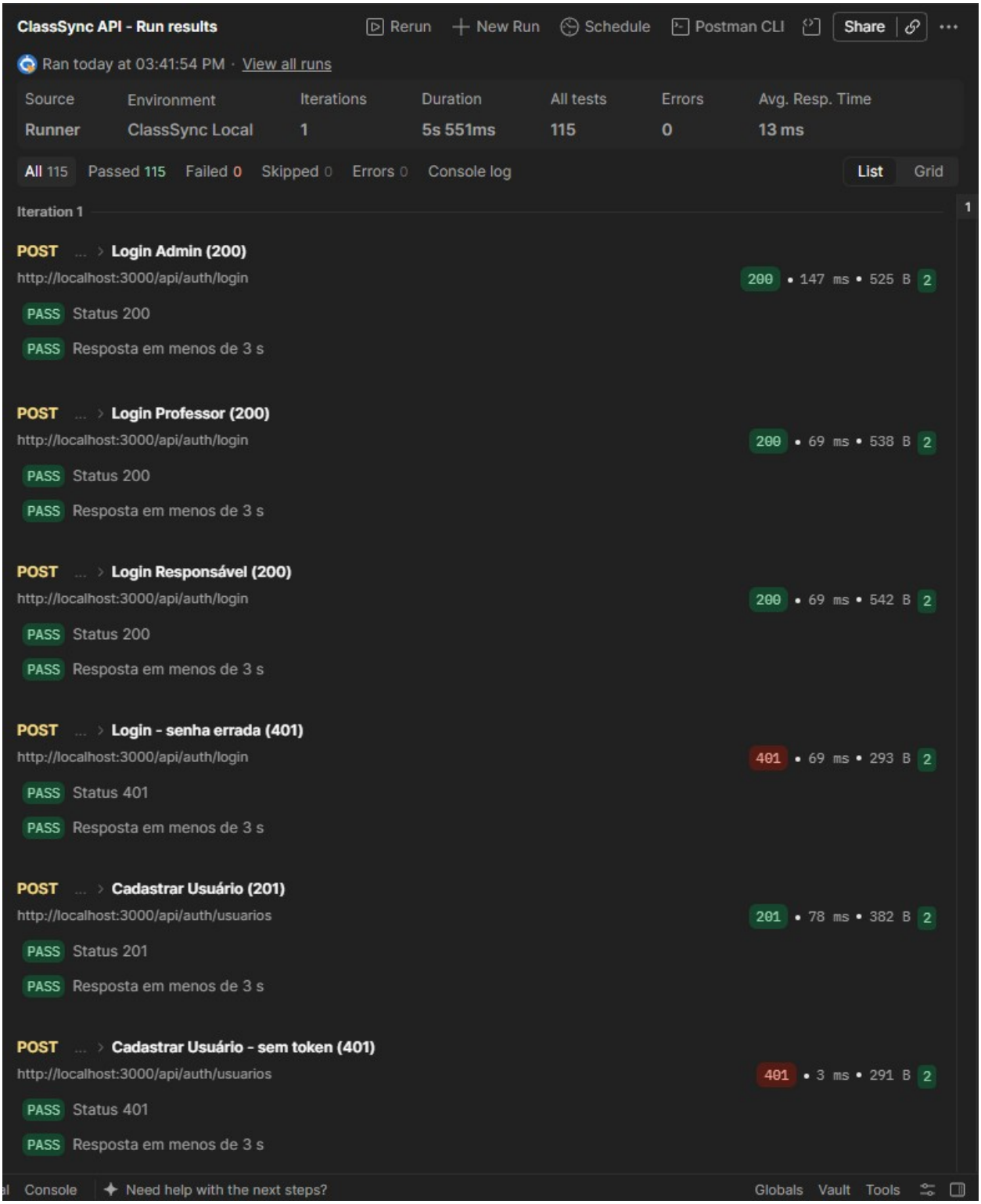
#	Requisição	Método e rota	Perfil	Esperado	O que comprova
40	Admin Devolvendo com motivo curto	PATCH /acompanhamentos/{id}/devolver	Admin	400	Devolução exige motivo com tamanho mínimo
41	Admin Devolvendo Certo	PATCH /acompanhamentos/{id}/devolver	Admin	200	Registro volta ao professor com o motivo
42	Professor Editando Devolvido	PUT /acompanhamentos/{id}	Professor	200	Devolvido pode ser corrigido
43	Professor Reenviando	PATCH /acompanhamentos/{id}/enviar	Professor	200	Correção volta para a revisão
44	Responsável antes da publicação	GET /acompanhamentos	Responsável	200	Registro ainda não aparece para a família
45	Publicar - professor	PATCH /acompanhamentos/{id}/publicar	Professor	403	Só o Admin publica
46	Publicar - admin	PATCH /acompanhamentos/{id}/publicar	Admin	200	Registro passa a Publicado
47	Responsável depois da publicação	GET /acompanhamentos	Responsável	200	Agora o registro aparece para a família
48	Filtro por Status	GET /acompanhamentos?status=PUBLICADO	Admin	200	Filtro por situação
49	Status Inválido	GET /acompanhamentos?status=XYZ	Admin	400	Filtro aceita só status válidos
50	Excluir - admin	DELETE /acompanhamentos/{id}	Admin	200	Remove o registro de teste, e a coleção pode rodar de novo

Ciência

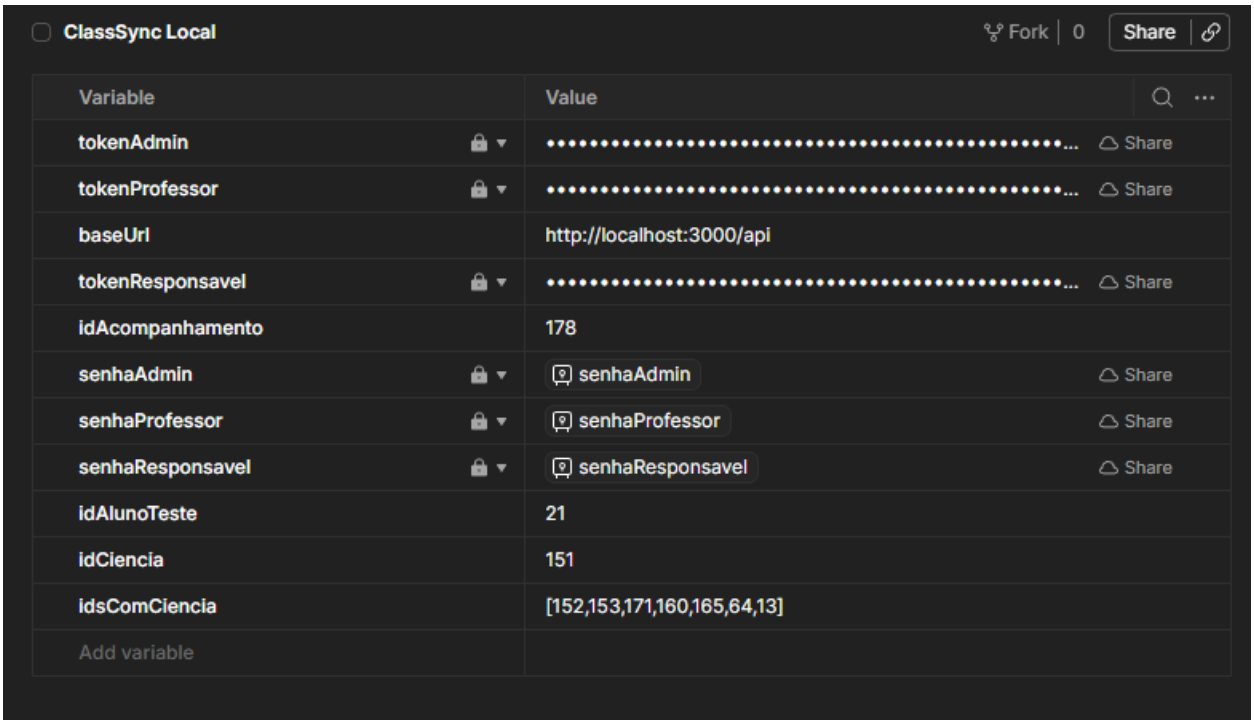
#	Requisição	Método e rota	Perfil	Esperado	O que comprova
51	Retornos da família	GET /retornos	Responsável	200	Lista as ciências já registradas pela família
52	Relatórios publicados da família	GET /acompanhamentos	Responsável	200	Escolhe um relatório publicado ainda sem ciência
53	Registrar Ciência	POST /acompanhamentos/{id}/ciencia	Responsável	201	Responsável confirma que leu o relatório
54	Registrar Ciência Duplicada	POST /acompanhamentos/{id}/ciencia	Responsável	409	Ciência é registrada uma vez só
55	Registrar Ciência - Professor	POST /acompanhamentos/{id}/ciencia	Professor	403	Ciência é papel do responsável
56	Registrar Ciência de outra família	POST /acompanhamentos/1/ciencia	Responsável	404	Não se registra ciência em relatório de outro aluno

Evidências

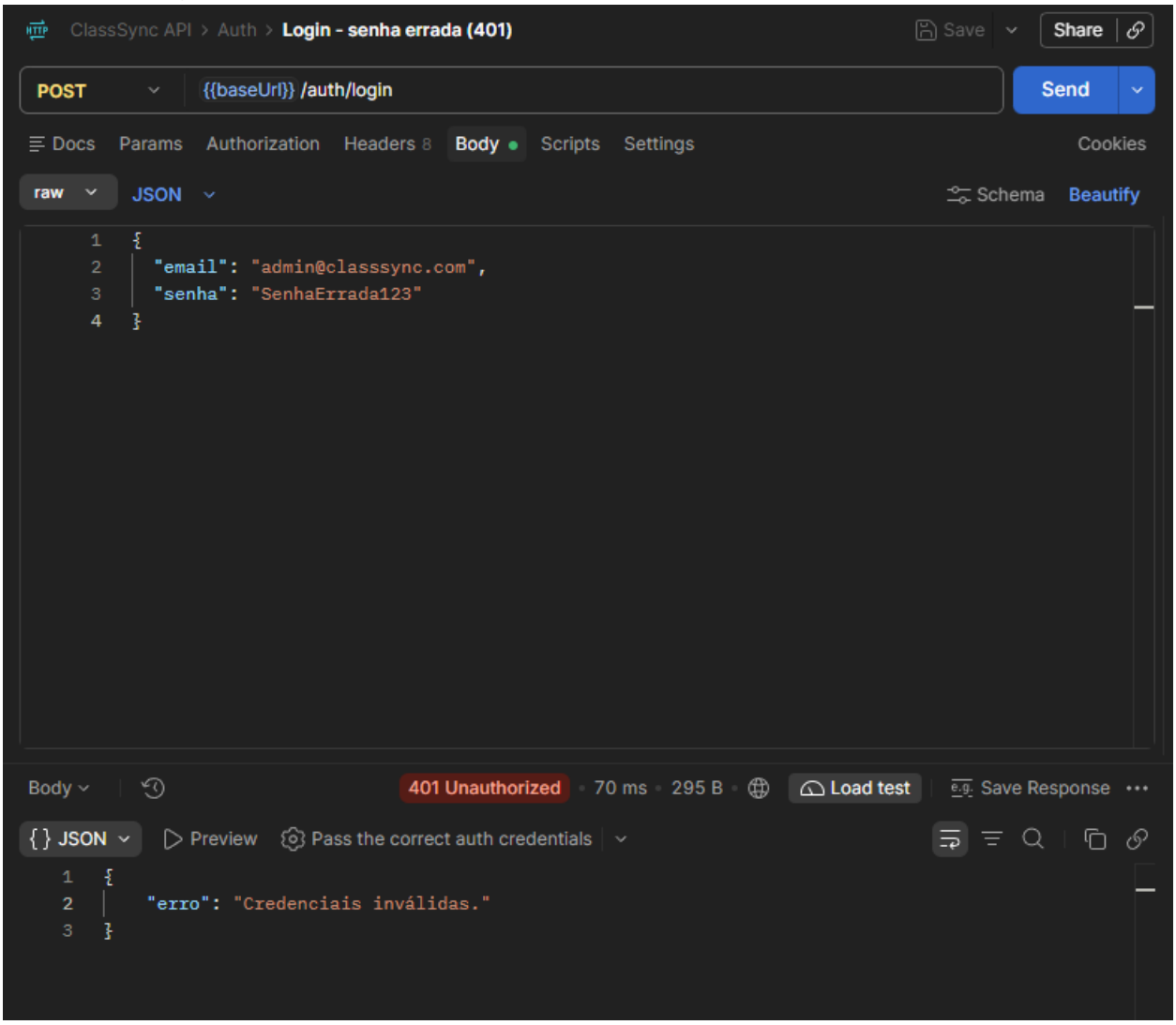
- **Print 1 — Collection Runner:** tela final com todas as requisições e o total de testes aprovados



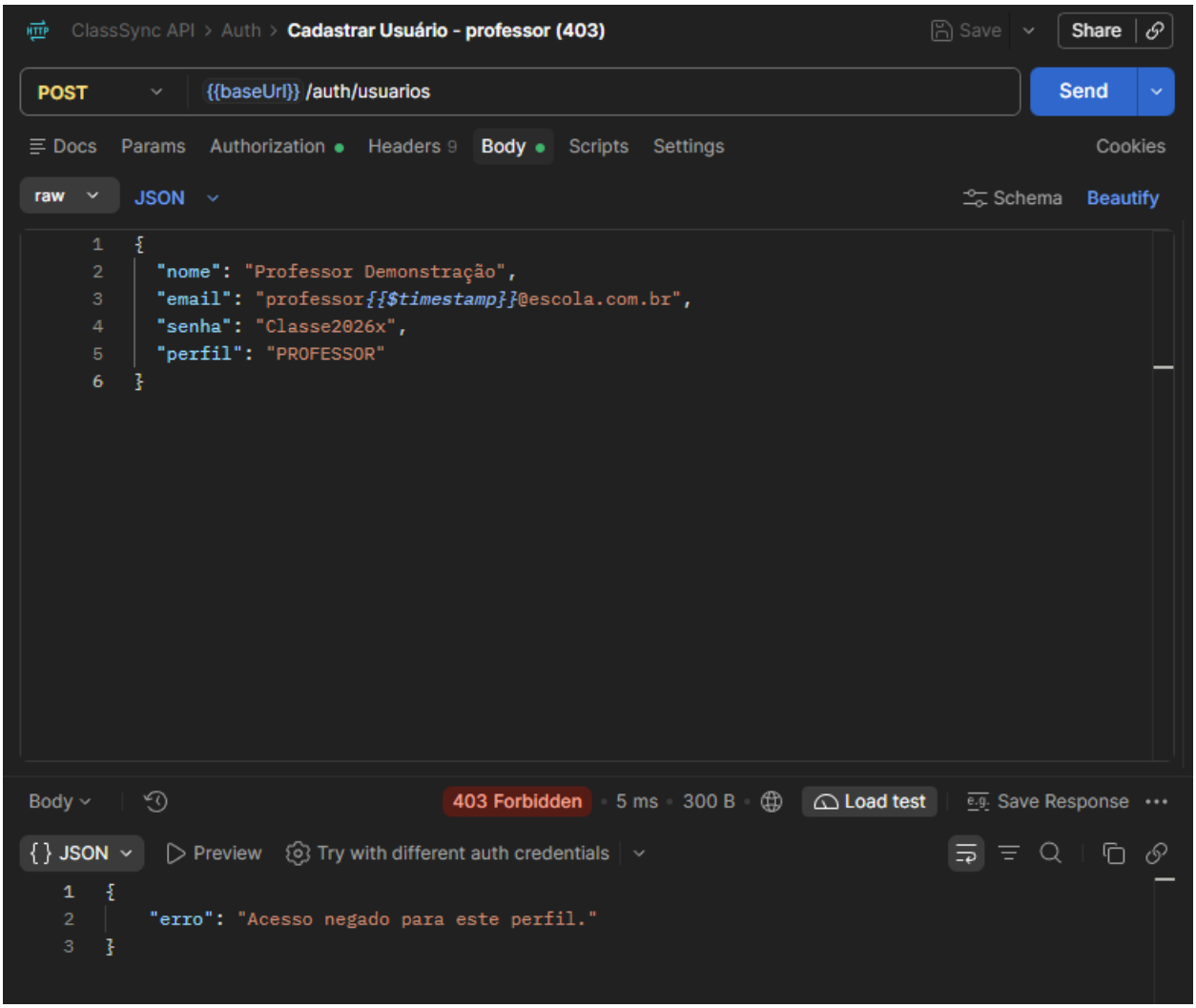
- **Print 2 — Environment:** a baseUrl fica visível, porque é só o endereço do servidor local. As senhas e os tokens são secret e aparecem ocultos.



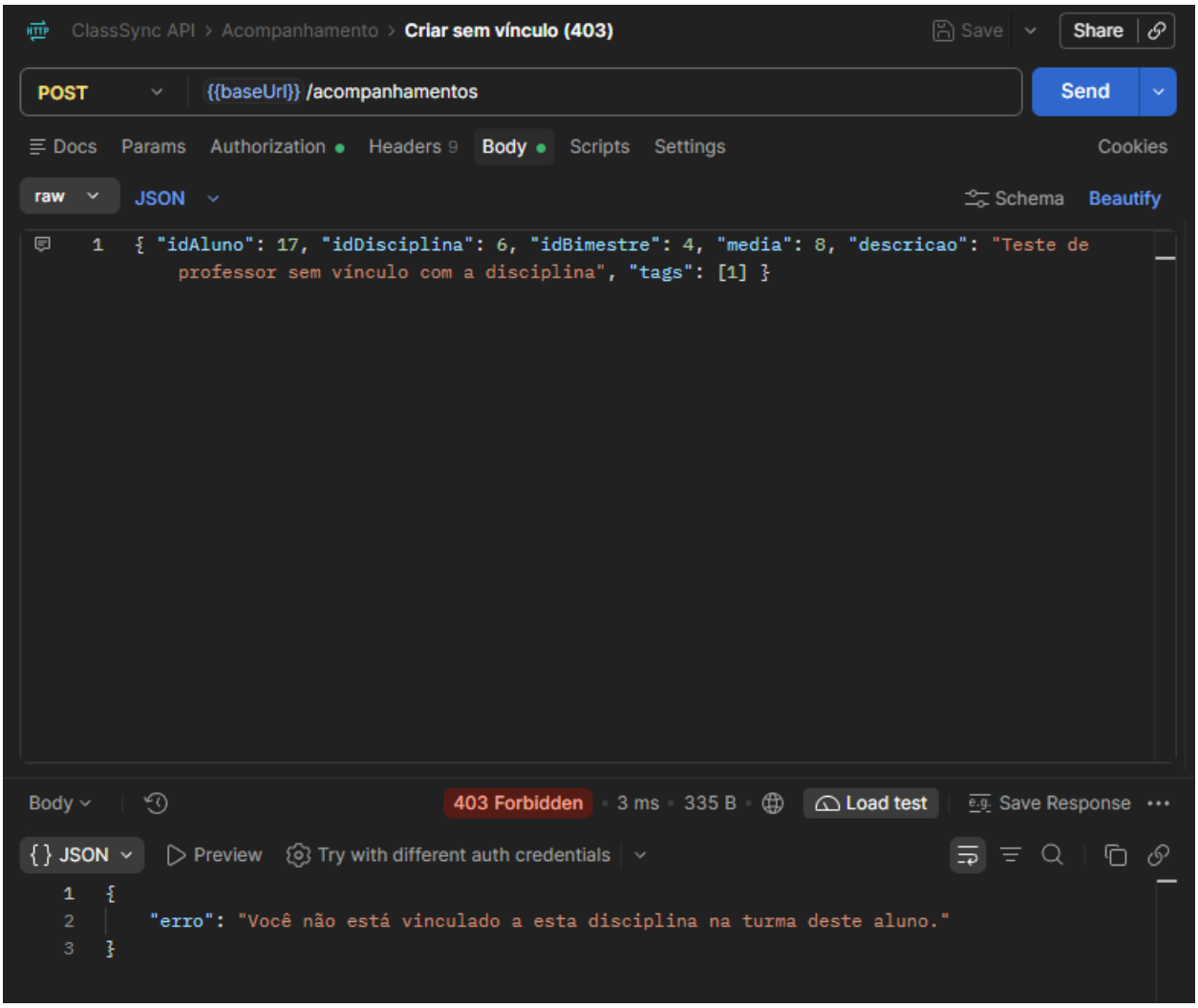
- **Print 3 — #4 Login - senha errada (401):** autenticação recusa credenciais erradas.



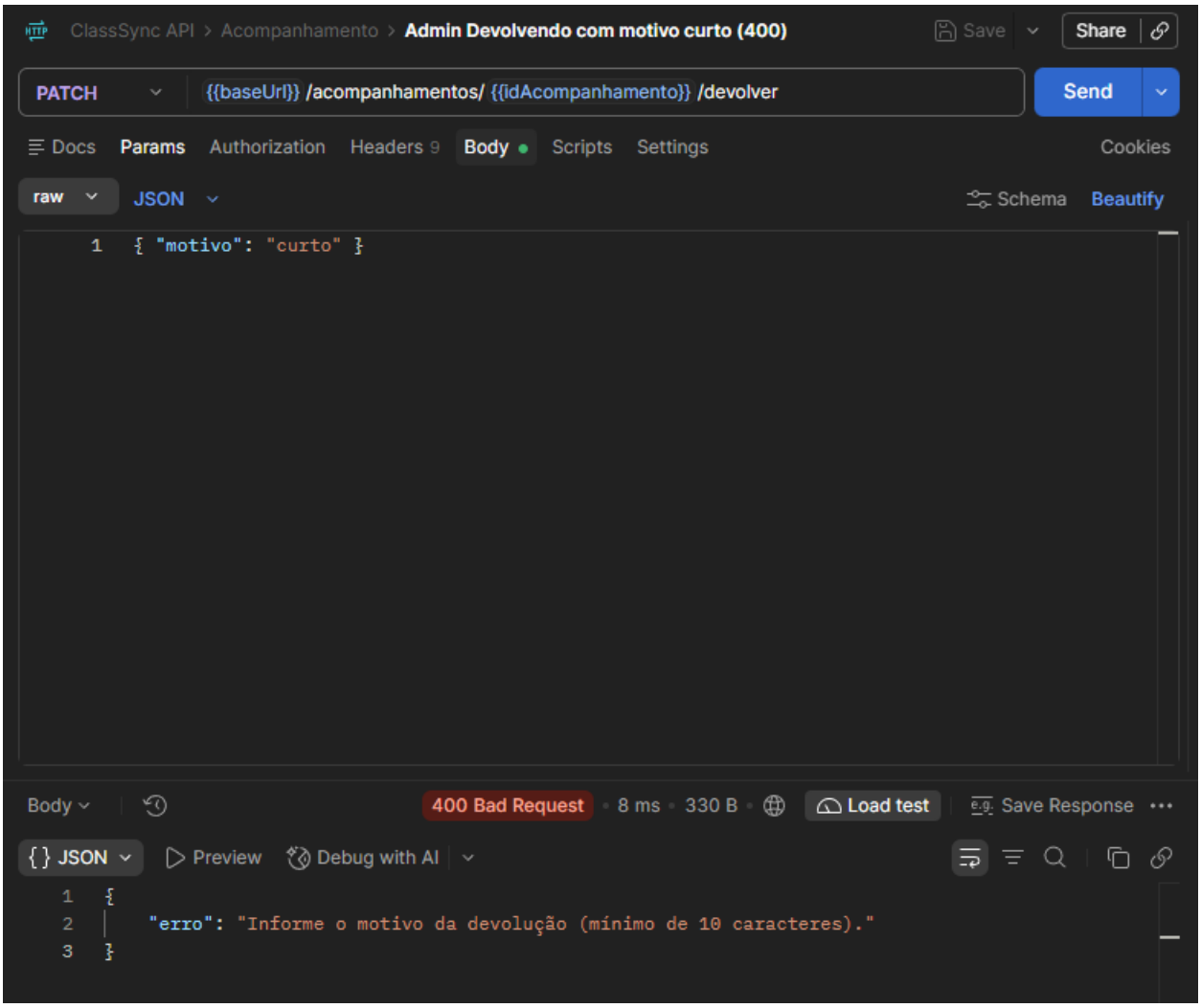
- **Print 4 — #7 Cadastrar Usuário - professor (403):** permissão por perfil.



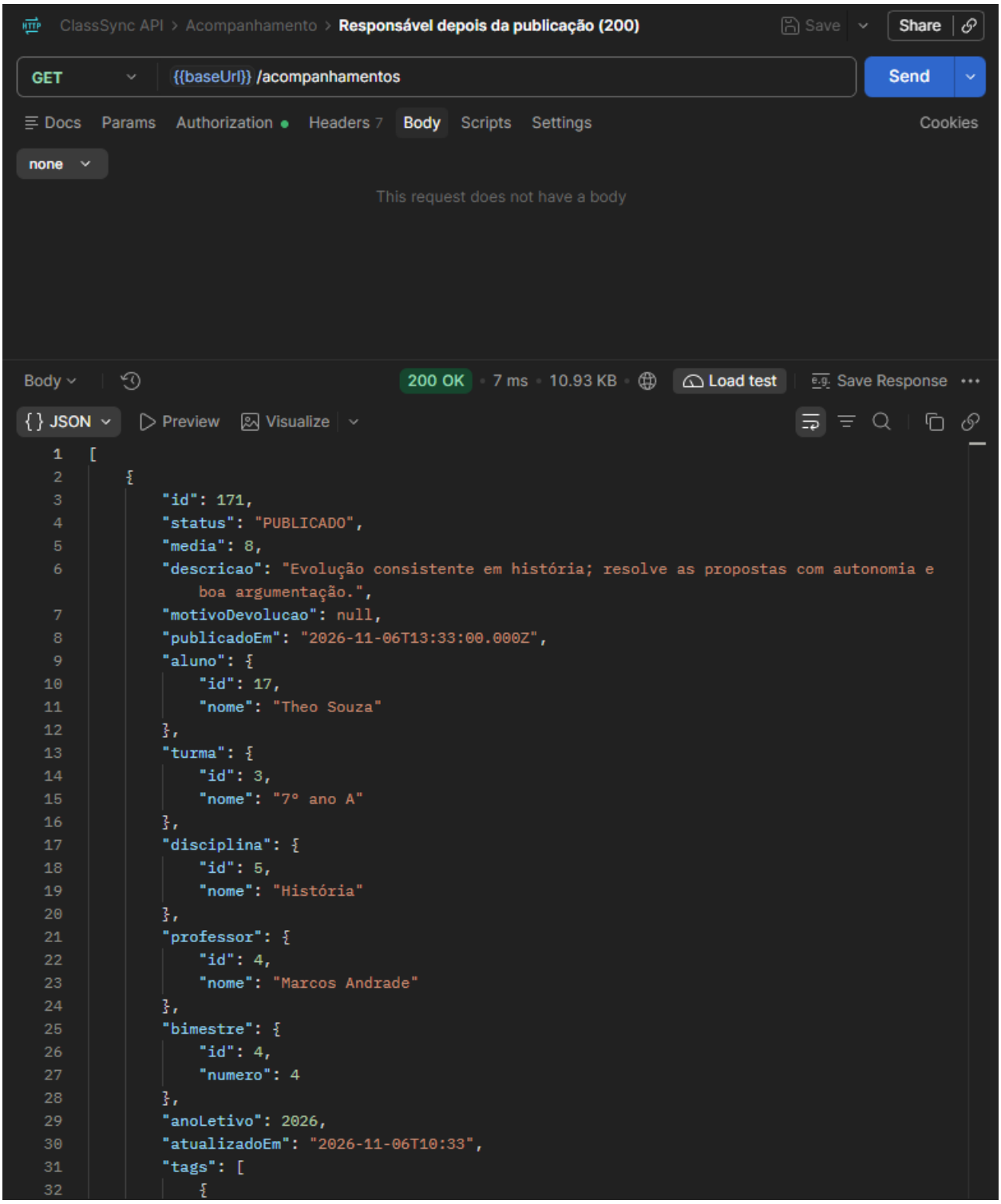
- **Print 5 — #31 Criar sem vínculo (403):** professor só registra onde leciona.



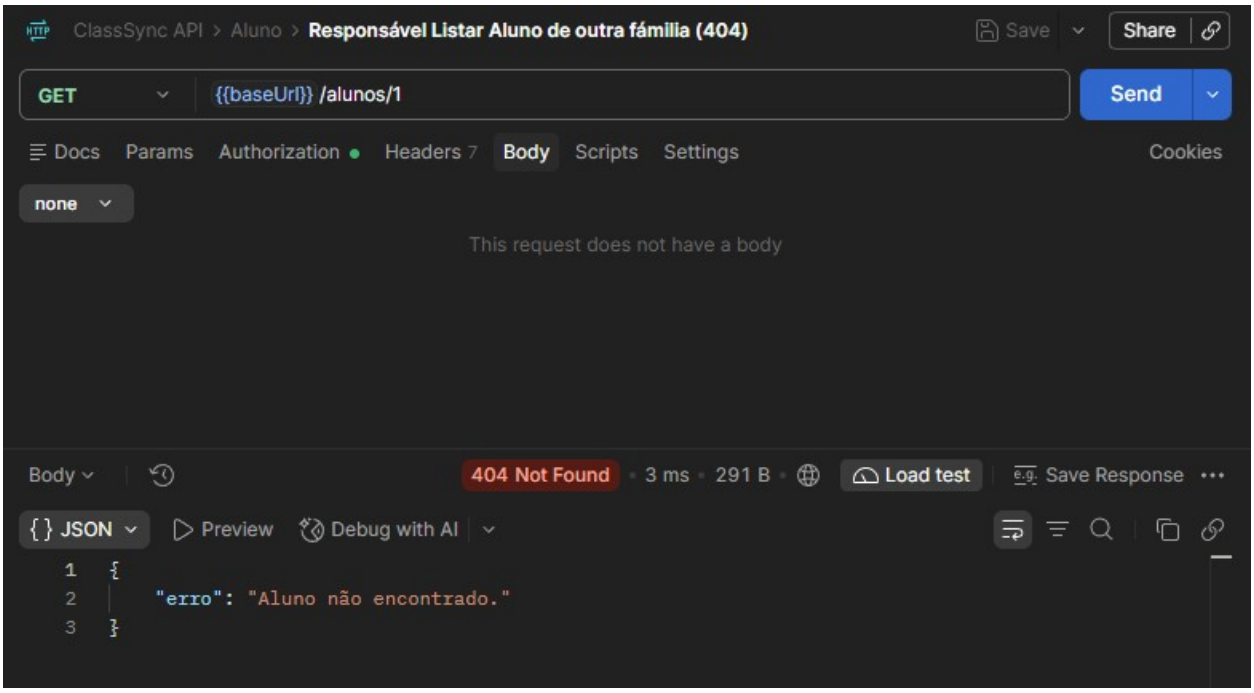
- **Print 6 — #40 Admin Devolvendo com motivo curto (400):** validação de regra de negócio.



- **Print 7 — #47 Responsável depois da publicação (200):** o relatório só aparece para a família depois de publicado.



- **Print 8 — #21 Responsável Listar Aluno de outra família (404):** proteção dos dados de outras famílias (LGPD).



Recuperação de senha (testada em produção)

As rotas de recuperação de senha foram testadas em 09/11/2026 no ambiente de produção (API no Render, banco no Aiven e e-mail real enviado pelo Brevo), pela tela do sistema e por requisições feitas no navegador. Os testes 57, 62, 63 e 64 foram aprovados: o e-mail chegou com o link, a nova senha foi gravada, o mesmo link foi recusado na segunda tentativa e o login funcionou com a nova senha. Os testes 58 a 61 entram na coleção do Postman na próxima etapa.

#	Requisição	Método e rota	Esperado	O que comprova	Resultado
57	Solicitar com e-mail cadastrado	POST /auth/recuperar-senha	200	Envia o e-mail com o link de redefinição	**Aprovado**
58	Solicitar com e-mail não cadastrado	POST /auth/recuperar-senha	200	Mesma resposta do teste 57: não revela quem tem conta	Próxima etapa
59	Solicitar com e-mail inválido	POST /auth/recuperar-senha	400	Valida o formato do e-mail	Próxima etapa
60	Redefinir com token inválido	POST /auth/redefinir-senha	400	Link adulterado ou expirado não funciona	Próxima etapa

#	Requisição	Método e rota	Esperado	O que comprova	Resultado
61	Redefinir com senha fraca	POST /auth/redefinir-senha	400	Senha segue a regra: 8 caracteres, com letras e números	Próxima etapa
62	Redefinir com token válido	POST /auth/redefinir-senha	200	Grava a nova senha	**Aprovado**
63	Redefinir com o mesmo token de novo	POST /auth/redefinir-senha	400	O link vale uma vez só	**Aprovado**
64	Login com a nova senha	POST /auth/login	200	A nova senha funciona	**Aprovado**